

Clustering

Krishna Sekhar

Padma Venkatraman

From PCA to clustering

- PCA found the directions of maximum variance — useful for reducing dimensions and spotting dominant trends
- But what if the structure in your data is **groups**?
- PCA can reveal clusters (points separate in PC space), but it doesn't *identify* them — it doesn't tell you which points belong together

We need algorithms that find groups automatically:
clustering.

What is clustering?

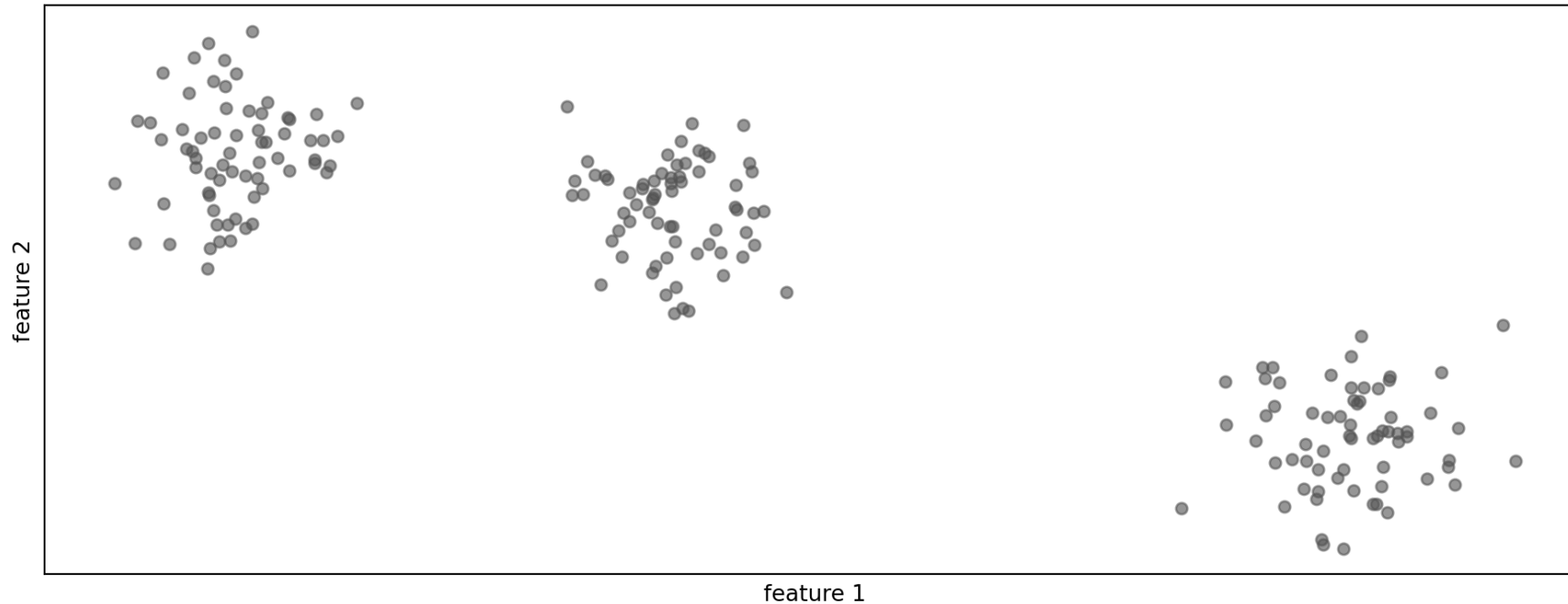
- Partition data into groups where points within a group are more similar to each other than to points in other groups
- *Unsupervised*: no labels, no “right answer” — the algorithm decides what counts as a group
- Different algorithms have different definitions of “similar”

The simplest and most widely used: *K-means*.

Not to be confused with *k*-nearest neighbours — k-NN needs labelled data (*supervised*), k-means finds its own groups (*unsupervised*). Same *k*, different job.

You can see the groups

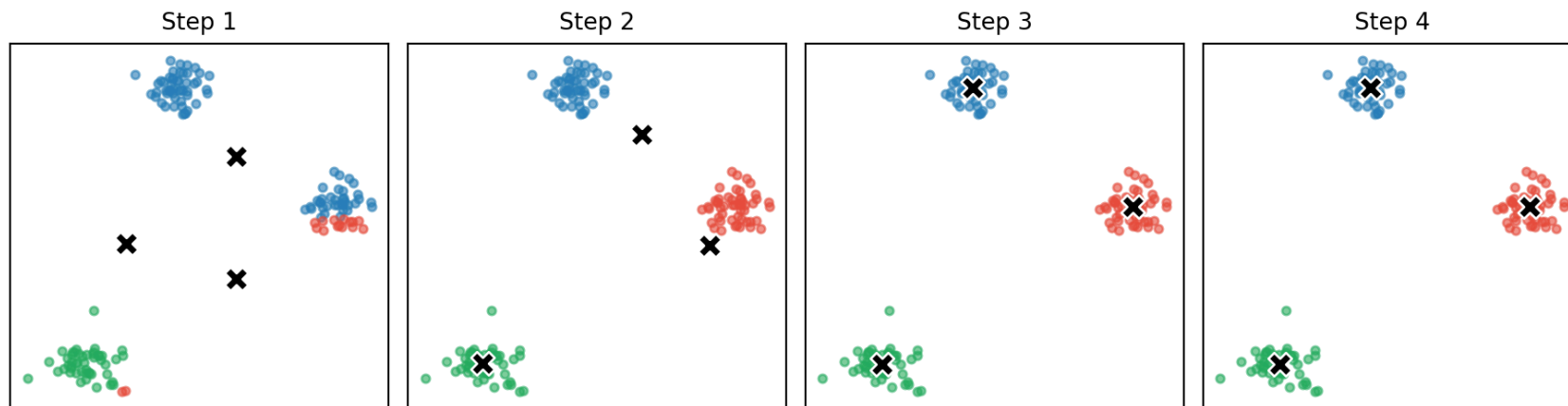
Where would you draw the boundaries?



Your eye finds the groups instantly. How do we setup an algorithm to do the same?

K-means — the idea

1. Pick K and place K random *centroids* (cluster centres)
2. **Assign** each point to its nearest centroid
3. **Update** each centroid to the mean of its assigned points
4. Repeat until assignments stop changing



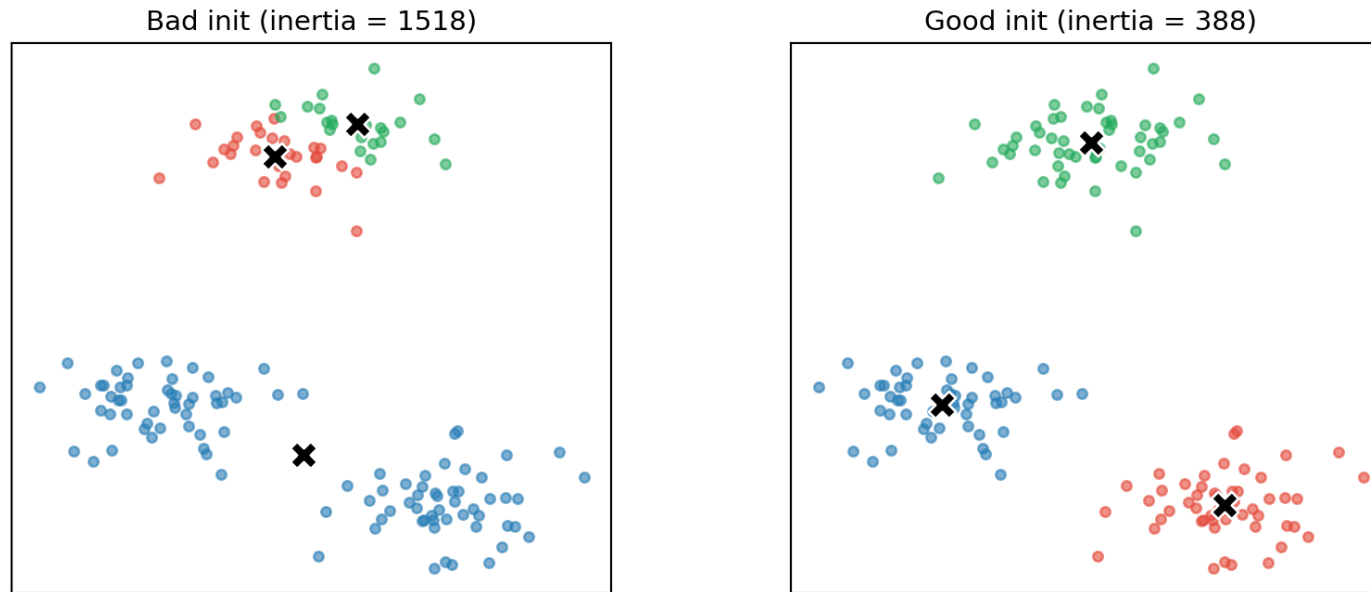
K-means — convergence

K-means minimises the *inertia* (total within-cluster distance):

$$\text{Inertia} = \sum_{i=1}^n \|x_i - \mu_{c(i)}\|^2$$

Both the assign and update steps can only decrease inertia, so K-means always **converges**. But it converges to a *local minimum*, not necessarily the global one.

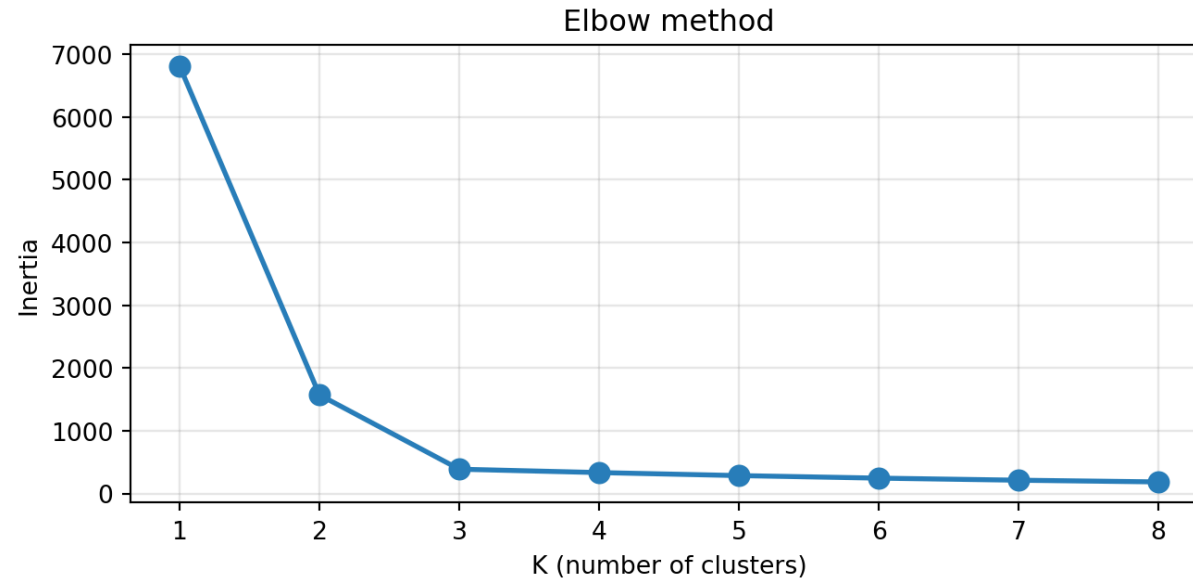
Initialization matters



Different starting positions → different local minima → different clusterings. Fix: run K-means multiple times with random inits, keep the lowest inertia.

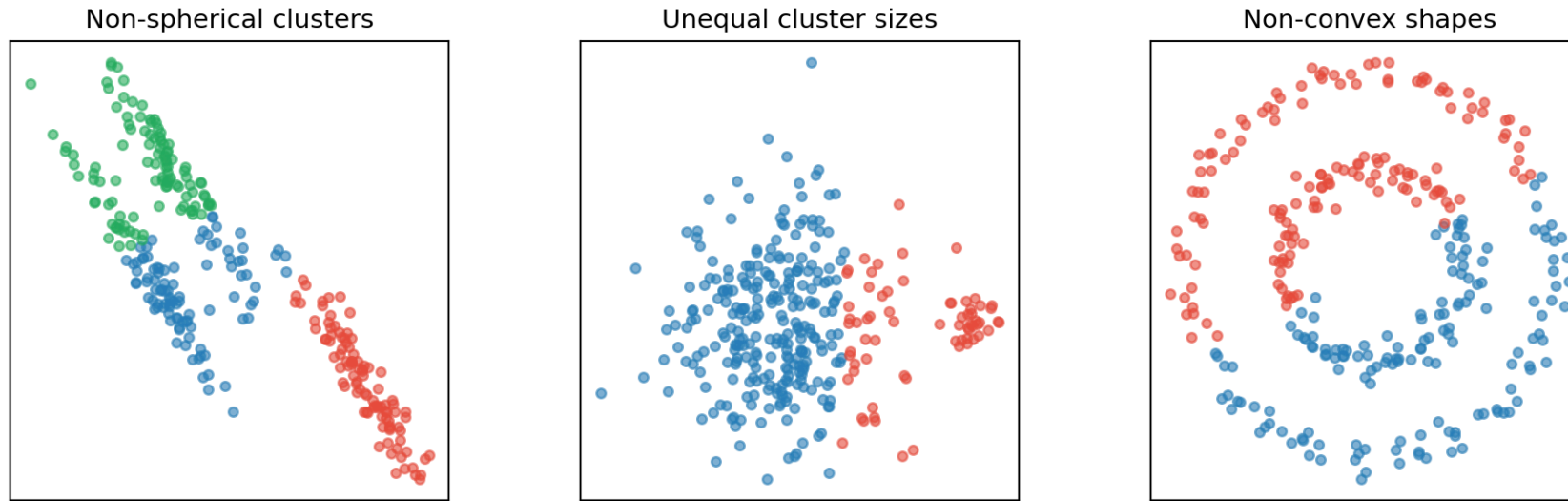
Choosing K — the elbow method

Plot inertia vs K and look for where it flattens — same idea as the scree plot.



Same subjectivity as scree plots — the elbow is a heuristic, not a guarantee.

K-means failure modes

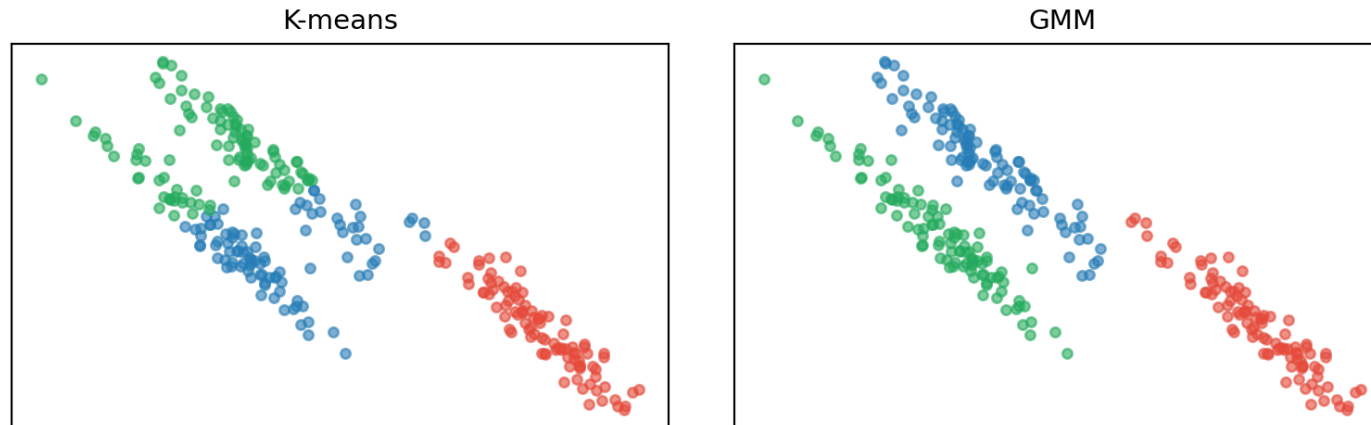


K-means assumes clusters are **spherical and roughly equal-sized**. When that's wrong, it splits or merges clusters in ways that don't match the real structure.

Aside: Gaussian Mixture Models

GMM is a probabilistic cousin of K-means:

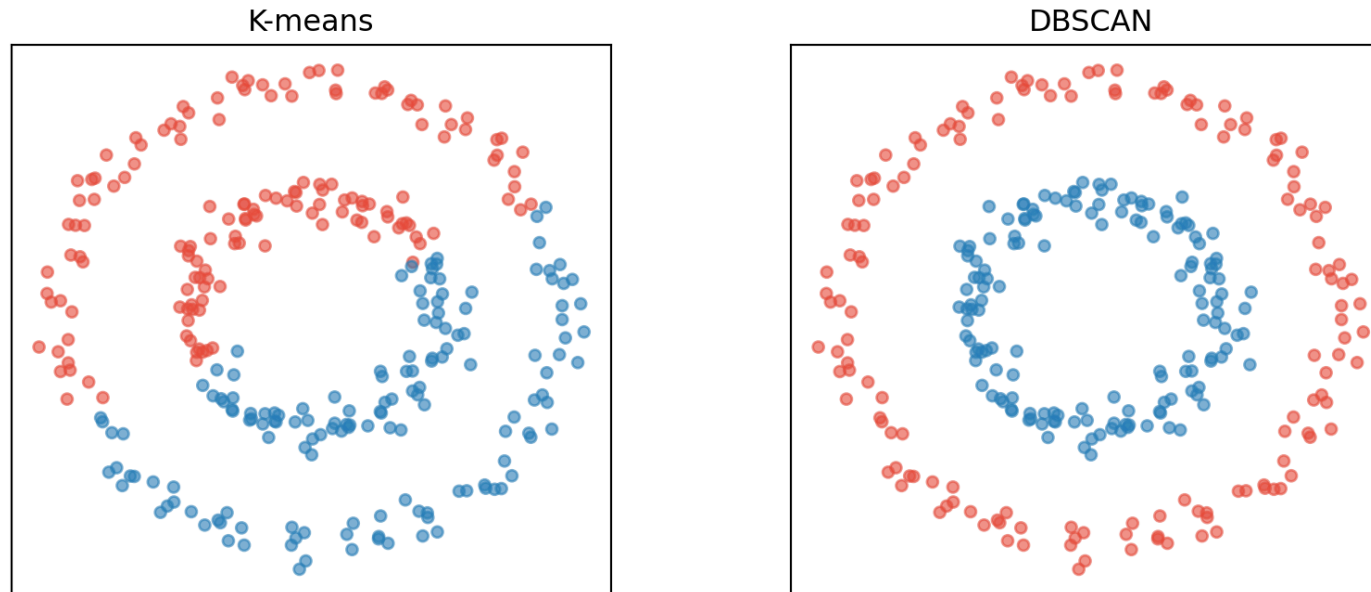
- *Soft assignments*: each point gets a probability of belonging to each cluster, rather than a hard label
- Each cluster has its own covariance — can capture elongated or tilted shapes
- Similar failure modes: still needs K , sensitive to initialization



The same elongated clusters K-means just failed on — GMM's per-cluster covariance recovers them.

Preview: DBSCAN

K-means needs you to choose K , and it can't handle the concentric rings we just saw. What if there's an algorithm that figures out the number of clusters on its own?



DBSCAN finds clusters by following regions of high density. No K needed, arbitrary shapes, and it naturally identifies outliers. We'll explore how in the next session.

Summary

- **K-means**: assign-update loop, always converges (to a local minimum)
- **Choosing K**: elbow method — same subjectivity as scree plots
- **Failure modes**: non-spherical, unequal sizes, non-convex shapes
- **GMM**: soft assignments, more flexible shapes, similar limitations
- **Next**: DBSCAN — density-based, no K , arbitrary shapes